

Tập 4 - DNS

Vạch trần lãng phí bằng thông & 3 kỹ thuật tối ưu DNS K8s

Phần 0 — Nền tảng K8s Networking · #DNS #CoreDNS #ndots #Headless



Lộ trình bài học: Lab First - Slide Second

Hôm nay chúng ta sẽ phơi bày một "sự lãng phí ngầm" trong mạng K8s và cách khắc phục:

- **PHẦN 1: Thực hành bắt quả tang (Lab First)**

- Dùng `tcpdump` nghe lén lưu lượng DNS bên trong Pod.
- Đếm số lượng queries thừa khi gọi API bên ngoài (Internet).
- Thử nghiệm 3 cách cấu hình để triệt tiêu queries rác.
- So sánh dịch DNS của Service thường với Headless Service.

- **PHẦN 2: Giải phẫu cơ chế (Slide Second)**

- Phân tích file cấu hình `/etc/resolv.conf` của Pod.
- Giải thích cận kề thông số `ndots:5` và cơ chế search paths.
- Đúc kết cẩm nang tối ưu mạng DNS cho môi trường Production.

PHẦN 1: Bắt đầu làm Lab ngay!

Chúng ta sẽ thực hiện các Thí nghiệm đột phá trong file `lab-guide.md`:

1. **Thí nghiệm 1:** Chạy `tcpdump` nghe lén UDP port 53 trên `pod-a`. Chạy curl tới `httpbin.org`. Chứng kiến cảnh 1 lần curl gửi ra tận **4 queries DNS** rác!
2. **Thí nghiệm 2:** Tối ưu DNS bằng FQDN — thêm dấu chấm cuối tên miền: `curl httpbin.org.` và chứng kiến số lượng query giảm về 1.
3. **Thí nghiệm 3:** Ép cấu hình `ndots:2` bằng cách khai báo `dnsConfig` trong YAML của Pod.
4. **Thí nghiệm 4:** Tạo Headless Service (`clusterIP: None`) và chạy `nslookup` để thấy DNS trả về trực tiếp danh sách IP thật của Pod.

👉 Hãy tạm dừng video, mở terminal và bắt đầu gõ lệnh trong tệp `lab-guide.md` nhé!

PHẦN 2: Tại sao 1 lệnh curl lại gửi đi 4 truy vấn DNS?

Hãy xem ruột file `/etc/resolv.conf` bên trong Pod của bạn:

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

- Khi bạn gọi `httpbin.org` (tên miền chỉ có **1 dấu chấm**), vì số lượng dấu chấm nhỏ hơn quy định (`ndots:5`), Linux cho rằng đây là một tên miền viết tắt nội bộ.
- Nó sẽ tuần tự nối thêm các đuôi trong danh mục `search` để đi hỏi CoreDNS:
 - i. `httpbin.org.default.svc.cluster.local` (CoreDNS trả về NXDomain - Lỗi)
 - ii. `httpbin.org.svc.cluster.local` (Lỗi)
 - iii. `httpbin.org.cluster.local` (Lỗi)
 - iv. `httpbin.org.` (Chọc ra DNS ngoài - Thành công!)
- 🖱️ **Hậu quả:** CoreDNS bị quá tải, ứng dụng bị trễ thời gian phân giải DNS khi gọi API bên ngoài!

3 Cách Tối Ưu DNS trong Kubernetes

Cách 1: Sử dụng FQDN (Dấu chấm thần thánh ở cuối)

Gọi tên miền có dấu chấm cuối: `httpbin.org.`

Dấu chấm này báo với hệ điều hành: *"Đây là tên miền tuyệt đối (Fully Qualified Domain Name). Trông tôi có vẻ ngẩn nhưng tôi đã hoàn chỉnh rồi, đừng nối đuôi linh tinh nữa!"*. Linux sẽ bỏ qua toàn bộ search paths và chọn thẳng ra Internet.

Cách 2: Cấu hình giảm `ndots` qua `dnsConfig`

Cài đặt `ndots:2` cho Pod. Bất kỳ tên miền nào có từ 2 dấu chấm trở lên (ví dụ: `api.github.com`) sẽ lập tức được giải quyết trực tiếp mà không cần đi qua danh sách tìm kiếm nội bộ.

Cách 3: Headless Service — Địa chỉ thẳng đến Pod (không qua kube-proxy)

Đây là kỹ thuật khác với `ndots`: thay vì giảm số DNS queries, Headless Service (`clusterIP: None`) thay đổi *nội dung* trả về. CoreDNS trả về trực tiếp IP thật của từng Pod thay vì 1 ClusterIP ảo — client kết nối thẳng, bỏ qua kube-proxy hoàn toàn. Dùng cho StatefulSet (DB, Kafka, etcd) cần kết nối đến replica cụ thể.

Key Takeaways — Bài học cốt lõi

- **Sử dụng linh hoạt:**
 - Khi gọi dịch vụ **nội bộ**: Hãy dùng tên ngắn (ví dụ: `db`). Quy tắc `ndots:5` sẽ phát huy sức mạnh giúp tự điền đuôi `db.default.svc.cluster.local`.
 - Khi gọi dịch vụ **bên ngoài**: Bắt buộc phải thêm **dấu chấm ở cuối**, hoặc phải cấu hình giảm `ndots` trên file YAML của Deployment!
- **Hiệu năng Production**: Tối ưu DNS đúng cách có thể giúp cụm K8s lớn giảm từ 50% - 80% tải truy cập vào CoreDNS, ngăn chặn thảm họa nghẽn mạng do sập phân giải tên miền.

Tập tiếp theo: Kubelet gọi CNI để cắm mạng như thế nào? CNI thực chất là gì? Hãy tự tay đóng vai Kubelet cắm mạng thủ công bằng `cnitool`!